arXiv:submit/7683737 [cs.AI] 6 Jun 2026

Inno Agent: An Open-Source Personal Learning Agent with Layered Memory, Educational Post-Training, and Local Deployment

Hao Hao[1], Ye Lu[2], Ruotong Yang[3], Yongheng Guo[4], and Aimin Zhou[5]

Shanghai Institute of AI for Education

[1]hhao@mail.ecnu.edu.cn [2]51275901116@stu.ecnu.edu.cn [3]1095231462@qq.com
[4]1547615191@qq.com [5]amzhou@cs.ecnu.edu.cn

June 6, 2026

Code: `inno-agent` **Model:** `Qwen-Inno-35B-v1`

Abstract

General-purpose large language model (LLM) agents and coding agents perform strongly on software-engineering tasks, but they typically rely on large models, long context windows, and high inference cost. Education is a different optimization target: the task distribution is more structured, and the value lies in personalized explanation, misconception diagnosis, exercise generation, calibrated feedback, review scheduling, privacy, and low-latency continuous interaction. This report presents **Inno Agent**, an open-source personal learning agent that organizes long-term learning support into a layered memory system—an *L1 learner profile* of goals, knowledge states, misconceptions, and preferences; an *L2 native wiki knowledge base* that ingests, links, and maintains learning materials as browsable pages; and *L3 session records*—and grounds this system in the Dick and Carey systematic instructional-design loop over goals, learner and content analysis, objectives, strategy, materials, and formative evaluation. Rather than modifying the Pi coding-agent SDK, Inno Agent uses Pi's extension mechanism to connect learner memory, wiki retrieval, a proactive scheduler, multi-channel personal messaging, and a Practice Lab with a workspace-scoped web terminal to the agent runtime, injecting a short, decision-ready learner context pack before each turn instead of the full profile or raw transcript. The learner profile is treated as an *open learner model*: it is evidence-driven, updated by transparent rules, and inspectable and correctable through a dedicated UI. The resulting system is designed to be *RL-ready*: learner state, tool actions, feedback events, terminal outcomes, and session records are exposed as structured trajectories that can support later policy improvement. We also report an initial post-training run that combines educational data, general chain-of-thought distilled from Claude Opus, and de-identified Inno trajectories on the Qwen3.6 35B-A3B base model. The resulting Qwen-Inno-35B-v1 retains a comparable overall benchmark profile while substantially shortening its reasoning traces—median output length drops by 31.8% on AIME, 55.1% on MMLU-Pro, and 72.2% on HumanEval—which directly lowers decoding cost and user-visible latency. Model training and evaluation are therefore treated as early evidence for the system direction, while the main contribution of this report is the implemented learning-agent substrate for low-latency, privacy-sensitive, locally deployable education.

† The training and evaluation sections (sections 7 and 7.1 to 7.3) are preliminary: they report one supervised post-training run on Qwen3.6 35B-A3B with benchmark and reasoning-length analysis, while RL optimization and learning-outcome studies remain future work. All system claims (section 3–section 11) are grounded in the released source under `apps/inno-agent/`.

1 Introduction

Personalized tutoring is a long-standing goal in intelligent education because one-to-one instruction can substantially improve learning outcomes, yet it is difficult to deliver at scale [1, 28]. Classical intelligent tutoring systems made progress by maintaining explicit learner models of knowledge, mastery, and feedback policies, for example through knowledge tracing and model-based tutoring loops [2, 7, 21, 27]. Recent large language models (LLMs) broaden the interaction surface: they can explain concepts, generate examples, answer follow-up questions, and interact through tools [3, 29, 31, 26]. However, the educational problem is not only to answer a single question correctly. A useful learning agent must support a learner over time, remember evidence about goals and misconceptions, choose an appropriate teaching move, generate practice, schedule review, and remain responsive in privacy-sensitive environments.

This requirement creates a mismatch between general-purpose LLM agents and personal learning agents. Coding agents and tool-using agents are designed for open-ended software-engineering workflows: they inspect large codebases, execute commands, revise files, and solve long-horizon tasks [31, 12]. These systems are often paired with large models and long context windows, which improves generality but increases latency, inference cost, and data exposure. In education, the optimization target is different. The task distribution is more structured, and the quality criteria include pedagogical usefulness, misconception diagnosis, calibrated hinting, appropriate exercise generation, transparent personalization, spaced review, self-regulated learning support, and learner trust [6, 5, 4, 16, 20]. Simply wrapping a general coding agent as a tutor therefore inherits the cost profile of coding agents without necessarily providing the memory and feedback mechanisms required by learning.

The central technical challenge is to connect three forms of state that are usually treated separately. First, a tutor needs a durable learner model that stores goals, mastery estimates, misconceptions, preferences, and evidence [2, 4]. Second, it needs a knowledge memory that can ingest and query learning materials without relying only on opaque chat history or raw retrieval [14]. Third, it needs action traces from real learning activities, including tool calls, practice attempts, terminal runs, feedback events, and scheduled reviews. Long context alone is a poor substrate for this problem because it mixes durable learner facts with transient conversation, a limitation also observed in broader LLM-agent memory studies [22, 19, 11, 30]. Retrieval alone is also insufficient because it retrieves content but does not accumulate synthesis: the LLM re-derives structure at query time rather than maintaining a persistent, compounding artifact. The recent "LLM Wiki" pattern argues for an LLM-maintained wiki that incrementally integrates sources, updates entity and concept pages, records contradictions, and files useful answers back into the knowledge base [13]; several public projects and MCP/Obsidian implementations have already adopted this pattern in practice. For education, however, this wiki memory must be coordinated with a learner profile and dialogue history, because knowing the material is different from knowing how a particular learner is progressing. This motivates a system design in which memory, tools, and interaction traces are first-class components of the agent rather than incidental text in a prompt.

In this report we present **Inno Agent**, an open-source personal learning agent built around this design principle. Inno Agent uses Pi's extension mechanism [9] to connect learning-specific tools to an existing agent runtime without modifying the Pi kernel. The system organizes long-term support into three layers: an L1 learner profile for durable personalization, an L2 native wiki for learning materials and concept knowledge, and L3 session records for recent interaction and tool history. The L2 wiki is not treated as an optional skill or an external knowledge-base attachment; it is a native memory layer with its own ingestion, indexing, linking, query, and maintenance workflow. As shown in fig. 1, the implemented interface exposes this memory layer directly beside the learning

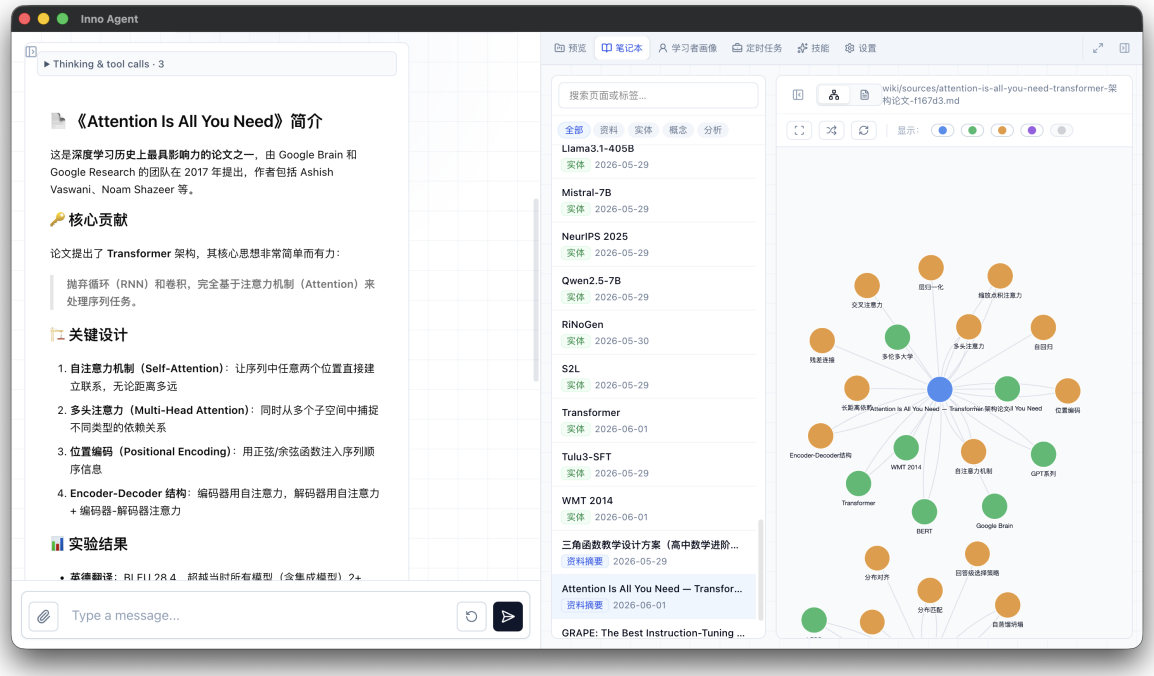


Figure 1: Inno Agent user interface with the L2 native wiki memory. The left panel shows a learning conversation, the middle panel lists wiki pages generated and maintained from learning materials, and the right panel visualizes the wiki graph over sources, entities, concepts, and analyses.

conversation, making wiki pages and their graph structure part of the learner-facing workflow. Around this memory stack, Inno Agent implements multi-channel personal messaging, a proactive scheduler, and a Practice Lab with a workspace-scoped web terminal. Before each turn, the system injects a short learner context pack instead of the full learner profile, so the model receives compact personalization while the durable state remains inspectable and editable.

This architecture also reframes model training. Rather than treating the model as the only carrier of educational intelligence, Inno Agent exposes an *RL-ready* substrate: learner state, tool actions, feedback, scheduled jobs, terminal outcomes, and session records can be logged as structured trajectories. These trajectories can support supervised fine-tuning and later reinforcement or preference optimization, following the broader lesson that model behavior can be shaped by feedback data and interaction traces [18]. Our current model work is intentionally preliminary: we have trained an initial educational-agent model based on Qwen3.6 35B-A3B and observed early signs of educational behavior and tool-use alignment, but systematic RL optimization and full quantitative evaluation remain future work. The main claim of this paper is therefore a system claim: educational agents need an explicit, instrumented learning substrate before model training and evaluation can be made reliable.

The motivation of this work is twofold. First, we study whether a three-layer memory system—learner profile, native LLM-maintained wiki, and dialogue/session records—can better satisfy the needs of personal learning than chat history, RAG, or a wiki tool alone. Second, we study whether a smaller, domain-specialized educational model, when supported by this explicit memory and tool scaffolding, can provide a practical alternative to deploying a general large coding agent for education. The expected advantages are lower latency, lower serving cost, stronger privacy through local or

organizational deployment, and better alignment with the repetitive but personalized structure of learning tasks. Inno Agent is designed as the system foundation for testing these hypotheses. This report makes the following contributions:

1. We design and release **Inno Agent**, an open-source personal learning agent extended from the Pi SDK [9]: learning-specific memory, scheduling, messaging, and Practice Lab tools are integrated through Pi's extension mechanisms rather than SDK-kernel changes.
2. We ground the system in the Dick and Carey systematic instructional design model [8], mapping instructional goals, learner/content analysis, performance objectives, strategies, materials, and formative evaluation to concrete agent components.
3. We implement a **layered memory architecture** with an L1 learner profile, an L2 native wiki knowledge memory, and L3 session records, treating the wiki as a native memory layer rather than as an optional skill or external knowledge-base attachment.
4. We implement an **evidence-driven learner profile** and a compact learner context pack, enabling personalization without placing the entire profile or raw history in every model context.
5. We implement a **learning-loop runtime** that combines proactive scheduling, multi-channel personal messaging, document/wiki ingestion, and a Practice Lab with terminal run records.
6. We make the system **RL-ready**† by exposing learner state, tool use, feedback, and practice outcomes as structured trajectories, and we report an initial Qwen3.6 35B-A3B training run with preliminary educational-agent results.

2 Background and Motivation

General-purpose coding agents. Coding agents couple an LLM with a tool loop: they read and edit files, run commands, inspect output, and plan across many steps. The capability that makes them effective on software engineering—long-horizon planning over a large, unfamiliar codebase—is also what makes them expensive, because it pushes toward the largest models and the longest context windows. Inno Agent reuses this *agent runtime* (the Pi coding-agent SDK) but does not assume that the *model* must be large, because the educational task it runs is narrower than general software engineering.

Educational agents require different optimization. An educational agent is judged on different axes than a coding agent. It must explain at the right depth, diagnose the cause of an error rather than only marking it wrong, offer a hint before a full solution, generate an exercise at an appropriate difficulty, plan a path, schedule review, keep its learner model transparent and correctable, and remain responsive across a long conversation. These goals favor an explicit learner model and a stable teaching policy over raw reasoning horsepower, and they favor low latency because tutoring is a tight, turn-by-turn interaction.

Why smaller domain-specialized models. Because educational tasks are more structured and more repetitive than software engineering, the teaching policy and the tool-use policy can plausibly be captured in domain data and distilled into a smaller model. A smaller model delivers more tokens per second on the same hardware, which directly improves the felt responsiveness of tutoring, and it fits deployment targets—a local GPU, a private server, a school's internal cluster—where keeping learner data inside an organizational boundary is a hard requirement rather than a preference.

Open learner models. A learner profile that influences teaching decisions should be inspectable and contestable by the learner. Inno Agent treats the profile as an *open learner model* [5, 4, 16]: the learner can review it, the system prompt forbids unevidenced labels, and the implementation exposes review and editing both as an agent tool (`review_learner_profile`) and as REST endpoints used by a dedicated UI panel. Profile conclusions are tied to recorded learning events rather than asserted directly, so a personalization judgment can be traced to the evidence that produced it.

Instructional systems design. The system is also motivated by instructional systems design, especially the Dick and Carey systematic design model [8]. The model treats instruction as a set of interdependent components: instructional goals are analyzed into prerequisite skills, learner and context characteristics shape the strategy, performance objectives determine assessment, materials operationalize the strategy, and formative evaluation feeds revision. This systems view is important for learning agents because it prevents personalization from becoming an isolated memory feature. A learner profile is useful only when it is connected to content analysis, success criteria, teaching moves, materials, and feedback evidence. The broader instructional-design literature makes the same point from complementary angles: Gagné connects learning outcomes to conditions of learning and instructional events [10], Merrill argues that effective instruction centers on authentic tasks, activation, demonstration, application, and integration [17], and Reiser situates instructional design within the historical development of educational technology and instructional systems [24]. Inno Agent therefore uses Dick and Carey as a pedagogical grounding for the system design, while the software architecture remains an extension-based implementation over the Pi runtime.

3 Inno Agent System Overview

3.1 Overall Architecture

Inno Agent is a single-user system with four layers: user interfaces, an application layer, an agent runtime built on the Pi SDK, and the layered memory. Figure 2 sketches the flow; every box corresponds to concrete modules in the released `apps/inno-agent/` source tree.

3.2 Design Principles

The implementation follows a conservative extension-first design. Inno Agent does not fork or modify the Pi SDK kernel; it starts a Pi `AgentSession` through `src/agent/pi-runner.ts` and adds learning behavior through the `createInnoExtension` factory in `src/agent/inno-extension.ts`. That extension registers model providers, L1 learner tools, L2 wiki tools, scheduler tools, document parsing, Practice Lab creation, and optional Pi subagents through Pi's provider, tool, and event hooks. The consequence is that Inno Agent remains a learning-specific product layer over a general agent runtime, rather than a separate agent framework.

The memory design follows the same separation. Learning state is made explicit and inspectable instead of being left as implicit chat context: L1 learner state is maintained in `src/memory/learner/`, L2 is implemented as a native TypeScript wiki memory in `src/memory/l2/`, and L3 remains the Pi `SessionManager`'s responsibility. The system prompt turns this boundary into an operating rule: stable learning facts are recorded through learner tools, knowledge artifacts are archived through `l2_archive`, and recent dialogue stays in L3 rather than being copied into the learner profile. Channels, the scheduler, and the Practice Lab all reuse this memory substrate. IM adapters and the Web UI may create or switch Pi sessions, but they still serve one personal learning system rather

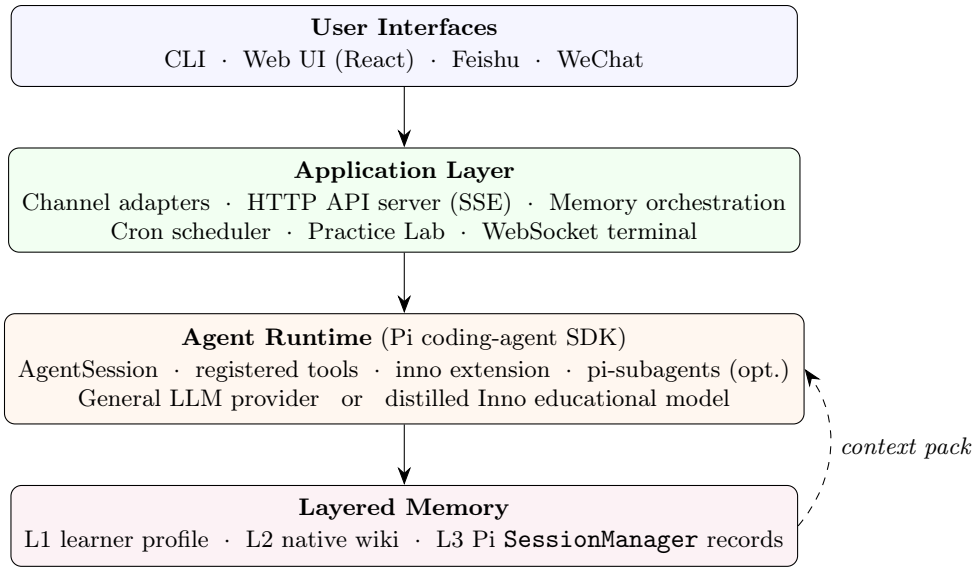


Figure 2: Inno Agent architecture. Learning behavior is added only through registered tools and one extension; the memory layer feeds a short context pack back into the runtime before each turn.

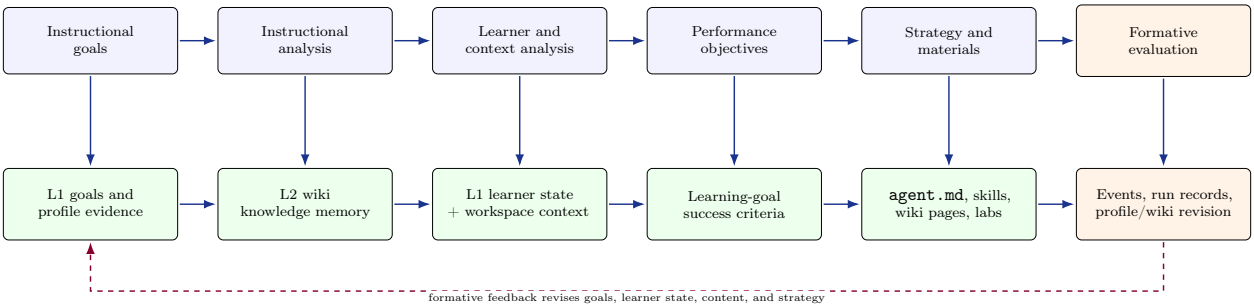


Figure 3: Pedagogical grounding of Inno Agent in the Dick and Carey systematic instructional design model. The upper row shows the instructional-design components; the lower row shows their implementation in Inno Agent. The dashed feedback path represents the formative-evaluation loop that updates learner profile state, wiki knowledge, workspace strategy, and later practice.

than a multi-tenant chatbot. The result is a closed learning loop in which conversation, memory updates, wiki maintenance, scheduled review, and practice runs reinforce one another.

Figure 3 makes the pedagogical mapping explicit. Inno Agent does not claim to automate the full work of a human instructional designer; instead, it operationalizes the main Dick and Carey components as persistent agent state and workspace-scoped procedures. L1 stores learning goals, learner characteristics, evidence, and success criteria; L2 stores content and prerequisite structure; `agent.md` and workspace skills encode local instructional strategy and material-production rules; practice tools and run records provide assessment evidence; and the profile/wiki/scheduler update loop implements formative revision.

3.3 Main Components

Table 1 summarizes the main components, but the system is best understood as one file-backed application composed around the Pi runtime rather than as a set of independent services. On the server

Table 1: Main components and their backing modules.

Component	Module	Responsibility
Agent runtime	`src/agent/`	Create Pi `AgentSession`, register tools, inject context
L1 learner memory	`src/memory/learner/`	Goals, knowledge states, misconceptions, preferences
L2 wiki memory	`src/memory/l2/`	Ingest, summarize, link, query a wiki knowledge base
L3 session memory	Pi `SessionManager`	Recent dialogue, tool calls, resume, compaction
Channel system	`src/channels/`	Feishu, WeChat entry points; unified dispatcher
Scheduler	`src/scheduler/`	Recurring and one-shot learning jobs
Practice Lab	`src/terminal/`, `src/agent/practice-tools.ts`	Workspace files, web terminal, run records
Web workspace	`web/src/`	Three-column React learning workbench

path, `src/server.ts` constructs the persistent stores and registries—`JobStore`, `ChannelRegistry`, `WorkspaceRegistry`, `RunRecordStore`, and `TerminalSessionManager`—then calls `initSession(...)` in `src/agent/pi-runner.ts`. On the CLI path, `src/cli.ts` runs Pi's `main(...)` with the same Inno extension factory. This keeps the CLI, Web UI, channels, scheduler, and Practice Lab on the same underlying memory and session model.

Figure 4 expands the table into the main turn-level flow. User-facing entry points create prompts or scheduled triggers; the Inno application layer selects the session and workspace; the Pi runtime executes a model turn with the Inno extension; tools then read or write the relevant memory layer before the response, push, wiki update, or run record is returned to the UI.

The agent runtime is the central integration point. `createInnoExtension` registers configured model providers from `src/config.ts`, then adds the learning tools that define the product surface: L1 tools such as `get_learner_context`, `record_learning_event`, `patch_learner_profile`, and `review_learner_profile`; L2 tools `l2_archive` and `l2_query`; scheduler tools for creating, listing, updating, deleting, and manually running jobs; document parsing through `parse_document`; and `create_practice_lab` when a workspace registry is available. Before each agent turn, the same extension handles `before_agent_start`: it loads the learner profile and recent learning events, builds a compact context pack, adds per-workspace `agent.md` and private `.skills` if present, appends the tail of the most recent run record, and finally appends Pi's own system prompt. This is the concrete mechanism by which learning state enters the model without modifying the Pi kernel.

The memory components differ in what they persist and when they are read. L1 stores a structured learner profile plus append-only learning events; `context-pack.ts` selects the active goal, weakest concepts, active misconceptions, teaching hints, due reviews, and recent event summaries for prompt injection. L2 implements the native wiki memory: `l2_archive` saves raw material, converts it into extracted Markdown, creates a source summary page, maintains linked entity/concept pages, rebuilds `wiki/index.md`, and appends `wiki/log.md`; `l2_query` reads the index and matched pages when answering questions about archived materials. L3 remains Pi's session layer in `paths.sessionDir`; Inno binds sessions to workspaces through `WorkspaceRegistry` rather than re-implementing dialogue history.

The application layer turns these memories into a learning workflow. Channel adapters implement a small `ChatChannel` interface, while `PersonalChannelDispatcher` maps Feishu and WeChat

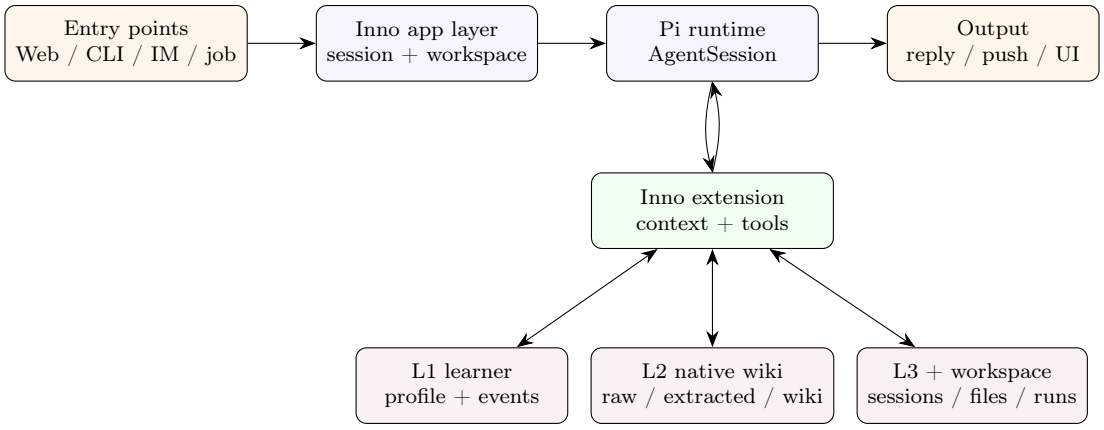


Figure 4: Turn-level component flow in Inno Agent. The Inno application layer routes entry points to a Pi session and workspace; the extension exchanges context and tool calls with the runtime; registered tools read from and write to L1, L2, and workspace state; and outputs return through the originating UI or channel.

private messages to Pi sessions, handles `/new` style commands, converts image/file attachments, records channel run logs, and preserves default push targets. The scheduler stores jobs under `src/scheduler/`; `CronScheduler` checks enabled jobs every 60 seconds and `executeJob` can push results back through the channel registry. Practice Lab is split between the agent tool and the terminal runtime: `create_practice_lab` writes code/data/README files into the current workspace, while `TerminalSessionManager` opens a workspace-scoped `node-pty` session, wraps Run-button commands with a sentinel, and stores outputs as run records.

The Web UI exposes the same structure to the learner. `web/src/react/App.tsx` implements the three-column shell: session sidebar, chat center, and workspace panel. `WorkspacePanel` hosts tabs for file preview, L2 notebook/wiki, learner profile, jobs, skills, and settings. The notebook UI loads wiki pages, filters them by page type, and visualizes the wiki graph; the workspace browser handles file preview/editing and the terminal drawer embeds xterm.js for interactive runs. Figure 5 shows the learner-profile view, where learning goals and knowledge states are editable next to the ongoing tutoring dialogue. The UI is therefore not only a chat wrapper: it is the inspection and control surface for the memory, workspace, scheduler, and Practice Lab components described above.

4 Layered Memory Architecture

4.1 Rationale

The three memory layers exist because learner state, archived knowledge, and recent dialogue have different lifecycles and different access patterns. Learner state (L1) is small, slowly changing, and read on every turn to set teaching depth; archived knowledge (L2) is larger, human-reviewable, and read on demand when a question touches stored material; recent dialogue (L3) is high-volume and transient, owned by the SDK's session manager. Mixing them would inject profile noise, make knowledge hard to audit, and inflate the prompt—exactly the failure mode the layering prevents. Explicit boundaries let a smaller model make personalization decisions from a short, stable context rather than re-reading long transcripts.



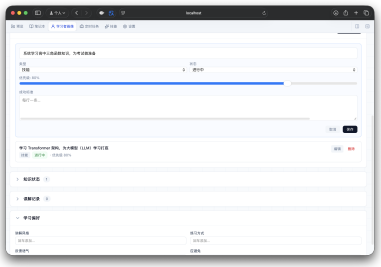
Figure 5: Learner profile view in the Inno Agent Web UI. The conversation remains visible on the left, while the workspace panel exposes recent learning events, learning goals, and editable learner-profile fields on the right. This makes L1 learner memory inspectable and correctable by the learner instead of leaving personalization hidden inside chat history.

4.2 L1: Learner Profile Memory

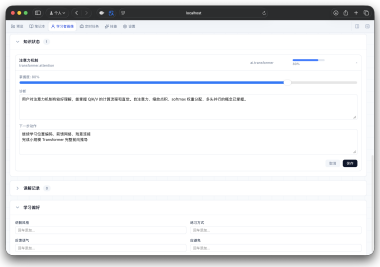
L1 answers the questions that set the teaching policy: what the learner wants to learn, what they have mastered, what they misunderstand, how they prefer to be taught, and what is due for review. The implemented profile schema is defined by the `LearnerProfile` interface in `src/memory/learner/types.ts` and mirrored by the web client in `web/src/types/learner.ts`. It is a deliberately focused subset:

```
export interface LearnerProfile {
  learner_id: string;
  version: number;
  updated_at: string;
  goals: LearningGoal[];
  knowledge_states: KnowledgeState[];
  misconceptions: Misconception[];
  preferences: LearnerPreferences;
  profile_summary: string;
}
```

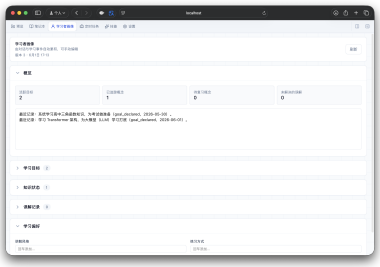
The richer learner-modeling dimensions discussed in the design notes (constraints, self-regulation, motivation/affect, and an explicit evidence index) are intentionally *not* top-level fields in the current implementation; the system tracks the four operational structures—goals, knowledge states, misconceptions, preferences—that directly drive a teaching move, plus a free-text `profile_summary`. The Web UI exposes these structures as an editable learner-facing model rather than a hidden personalization cache (fig. 6). Section 5 details these structures and their update logic.



(a) Profile overview and recent learning events.



(b) Editable knowledge state with mastery, diagnosis, and next actions.



(c) Editable learning goals and learner preferences.

Figure 6: L1 learner-profile memory as exposed in the Web UI. The panel makes the durable learner model inspectable and correctable: the learner can review recent profile evidence, inspect concept-level mastery and diagnosis, and edit goals or preferences that guide later tutoring turns.

4.3 L2: Wiki Knowledge Memory

L2 stores learning content as a native wiki rather than as opaque vectors. It archives source summaries, maintains concept/entity/analysis pages, and answers queries by reading an index and the relevant pages. Its on-disk layout under `data/l2/` is fixed by `wiki-maintainer.ts`:

```
data/l2/
|- raw/ # original uploads (read-only), by sub-type
|- extracted/ # faithful extracted markdown
|- wiki/
| |- index.md # rebuilt on each archive
| |- log.md # append-only maintenance log
| |- sources/ entities/ concepts/ analysis/
|- manifest.jsonl # append-only metadata index
```

Section 6 details the page schema and the ingestion and query pipelines.

4.4 L3: Session History

L3 is the Pi SDK's session layer. It holds recent dialogue and tool calls, supports session resume, and performs context compaction, so Inno Agent does not re-implement conversation memory. The application binds sessions to workspaces and, for messaging channels, allocates a dedicated session per chat so that channel traffic does not pollute the web session (section 9).

4.5 Memory Boundary Rules

The system prompt encodes the routing rule that keeps the layers clean: ability judgments go to L1, knowledge content goes to L2, and the live conversation stays in L3 (table 2).

5 Learner Profile Model

5.1 Learning Events

The L1 learner profile follows an evidence-first design: the system should remember learning-relevant evidence before it turns that evidence into a durable learner judgment, consistent with learner-modeling work that treats learner models as structured, evidence-bearing representations rather

Table 2: Memory boundary rules encoded in the system prompt.

Question	Memory layer
What does the learner want?	L1
Which concept is the learner struggling with?	L1
What notes or documents were archived?	L2
What did we discuss recently?	L3
What should the learner do next?	L1 + L2 + L3

than static labels [2, 4]. This prevents the profile from becoming a set of opaque labels such as "weak at programming" and instead keeps updates tied to concrete events: a declared goal, a practice attempt, an explanation, a self-assessment, an explicit preference, feedback, or a milestone. The implementation therefore separates three kinds of information. Factual observations are appended as learning events; inferred quantities such as mastery deltas or possible misconceptions are carried as optional `derived_signals`; and user-editable preferences remain in the profile so they can be corrected directly.

The event schema in `src/memory/learner/types.ts` is intentionally small and append-only. It records what happened, where it happened, which concepts or goals were involved, and any model-supplied hints that may help update the profile:

```
export type LearningEventType =
  | "goal_declared" | "exercise_attempt" | "concept_explained"
  | "self_assessed" | "preference_stated" | "feedback_received"
  | "milestone_reached";

export interface LearningEvent {
  event_id: string; // evt_<uuid8>
  learner_id: string;
  timestamp: string;
  event_type: LearningEventType;
  context: { goal_id?: string; concept_ids?: string[]; session_id?: string };
  payload: Record<string, unknown>;
  derived_signals?: {
    mastery_delta?: number;
    misconception_candidates?: string[];
    affect?: string;
    preference_candidates?: string[];
  };
}
```

Events are appended to `events.jsonl`; the derived profile is stored separately as `profile.json`, and `saveProfile` increments its `version` and `updated_at` timestamp on every save. The Pi extension registers `record_learning_event`, `patch_learner_profile`, `update_learner_profile`, `get_learner_context`, and `review_learner_profile` as L1 tools. The system prompt asks the agent to call `record_learning_event` for stable learning facts and goal changes, including the negative case where a learner stops or switches a goal; the tool appends the event and then calls `recordEventAndUpdateProfile` to fold deterministic signals into the profile. Figure 7 summarizes the resulting write, read, and correction paths.

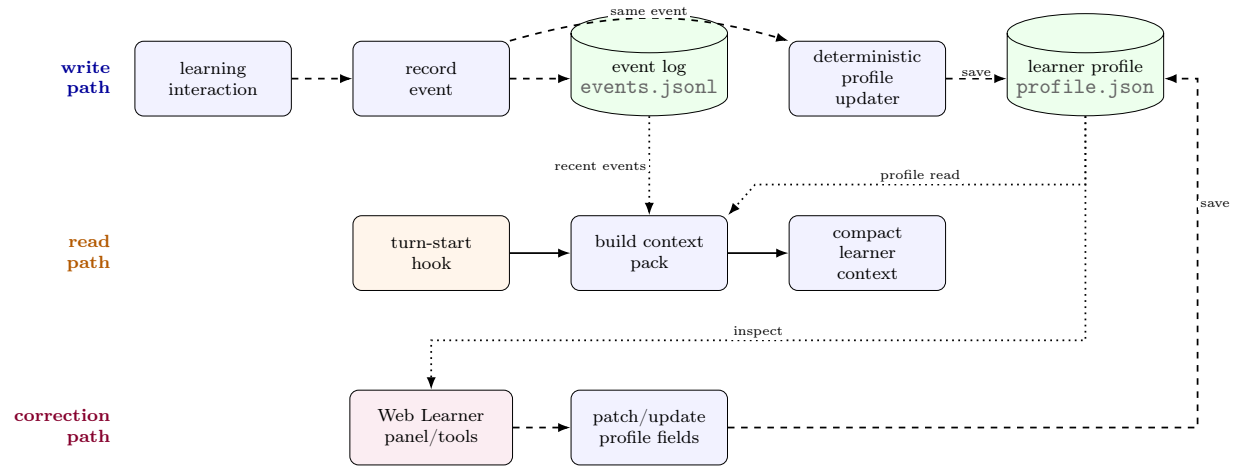


Figure 7: L1 learner-profile memory workflow. Learning events provide the write path into `events.jsonl`; deterministic update rules fold selected signals into `profile.json`; before each agent turn, the extension reads the latest profile and recent events to build a compact context pack. The Web UI and profile tools provide a correction path for the implemented learner-model fields.

5.2 Profile Schema

The design notes describe a broader learner model with constraints, learning behavior, self-regulation, motivation/affect, preferences, and a separate evidence index, reflecting common learner-model dimensions and self-regulated-learning concerns discussed in the education literature [2, 20]. The current implementation is deliberately narrower. As shown in section 4, the top-level profile stores goals, concept-level knowledge states, misconceptions, preferences, and a short textual summary. Evidence is not a separate top-level index in this release; instead, implemented structures carry local `evidence_ids` that point back to learning events.

Each `KnowledgeState` is concept-grained, following the same basic motivation as knowledge-tracing work: teaching decisions are more useful when they can refer to a particular knowledge component or concept rather than only to a whole course score [7, 21]:

```
export interface KnowledgeState {
  concept_id: string; concept_name: string; domain: string;
  mastery: number; // 0-1, clamped
  confidence: number; // 0-1, clamped
  stability: number; // 0-1, clamped
  last_practiced_at?: string;
  review_due_at?: string;
  evidence_ids: string[];
  diagnosis: string;
  next_actions: string[];
}
```

A `Misconception` adds a status lifecycle (`active` → `repairing` → `resolved` → `stale`), a severity and confidence, first/last-seen timestamps, supporting `evidence_ids`, and a `repair_strategy`. A `LearningGoal` carries a type (`skill` / `concept` / `project` / `exam` / `habit`), a priority, a status, success criteria, and a source. `LearnerPreferences` is four string lists—`explanation_style`, `practice_style`, `feedback_tone`, and `avoid`. This is a minimum sufficient memory for the current system: it stores only the fields that can change later teaching behavior, while leaving richer learner-modeling dimensions as future extensions. The full implemented schema is defined in `src/memory/learner/types.ts`.

5.3 Evidence-Driven Updates

Profile updates use transparent rules before heavier learner-modeling methods. This matches the design principle that an early personal learning agent has little per-user data, so simple, auditable updates are safer than claiming to implement Bayesian or deep knowledge tracing without enough longitudinal evidence [7, 23, 15]. In code, `applyLearningEventToProfile` (`auto-profile.ts`) composes goal, knowledge, preference, and summary updates and returns whether anything changed. It does not automatically promote every `misconception_candidate` or `affect` signal into a top-level profile field; those signals are available for tools or future update rules, while the implemented automatic path remains conservative.

The implemented knowledge update is intentionally small-step. If an event names one or more `concept_ids`, the updater creates a missing `KnowledgeState` with low initial mastery and confidence, appends the event id to `evidence_ids`, and adjusts mastery only if that event has not already been applied. Default mastery changes are event-typed: $+0.03$ for an exercise attempt, $+0.02$ for a concept explanation or milestone, and $+0.01$ for a self-assessment; a numeric `derived_signals.mastery_delta` can override the default. Confidence and stability are clamped to $[0, 1]$, and `review_due_at` is scheduled only for exercise attempts and concept explanations using the current mastery band ($< 0.4 \rightarrow +1$ day, $< 0.75 \rightarrow +3$ days, $\geq 0.75 \rightarrow +7$ days), a simple operational use of the broader spacing effect rather than a validated scheduling model [6]. Because event ids are checked before applying the mastery delta, replaying the event log does not repeatedly inflate mastery.

Goal and preference updates are similarly bounded. A `goal_declared` event can create or refresh a goal with default type `skill`, priority 0.8, status `active`, and source `user_declared`. If the event text expresses stopping or archiving a goal, matching goals are marked `archived` with priority 0, and related knowledge states have their next actions and review date cleared rather than being deleted. Preference candidates are mapped by simple string rules into operational hints such as `example_first`, `code_first`, `small_steps`, `immediate_feedback`, `encouraging`, or `socratic`. The free-text `profile_summary` keeps only a short tail of recent event sentences. The whole profile can be rebuilt from the event log by `rebuildProfileFromEvents`, which replays events with summary updates disabled and then regenerates the summary from the last eight event summaries.

5.4 Context Pack Generation

The profile is not injected wholesale into the model context. Before each turn, the Inno extension handles `before_agent_start` by loading the profile and the last eight learning events, building a compact context pack, formatting it as a "learning context" section, and prepending it to the system prompt. `buildContextPack` (`context-pack.ts`) selects the highest-priority active goal, the five weakest non-mastered concepts (ascending mastery), active misconceptions, teaching hints translated from preference codes, concepts whose `review_due_at` has passed, and up to five recent event summaries:

```
export interface LearnerContextPack {
  active_goal?: string;
  relevant_concepts: { concept_id: string; mastery: number; diagnosis: string }[];
  active_misconceptions: string[];
  teaching_hints: string[];
  recent_events?: { event_id; event_type; timestamp; summary }[];
  review_due_concepts?: { concept_id; review_due_at; mastery }[];
}
```

The same pack shape can also be written to `context-cache.json` when profile updates refresh the cache, but the current turn injection builds the pack directly from the latest profile and recent

events. This is the concrete mechanism behind the scaffolding hypothesis: the model receives a small, decision-ready summary instead of being asked to reconstruct learner state from a long transcript.

5.5 User Control

The learner profile is designed as an open learner model rather than a hidden personalization cache: prior work argues that learner-facing models should be inspectable, negotiable, and useful for reflection, not only internal to the tutor [5, 4, 16]. In the implemented system, `review_learner_profile` renders the current goals, knowledge states, misconceptions, preferences, and summary for inspection. The HTTP API exposes `GET/PATCH /api/learner/profile`, create/update/delete routes for goals, and PATCH routes for individual knowledge states and misconceptions; the web Learner panel uses these routes to edit the implemented fields. This realizes the inspect-and-correct part of the open-learner-model design for the current schema. Full long-term-memory deletion or opt-out controls are design goals rather than fully implemented profile-management flows in this release, so the paper treats them as future user-control work rather than completed functionality.

6 Native Wiki Knowledge Memory

6.1 Design Rationale

L2 implements a native wiki-style knowledge memory for durable educational content. In a standard retrieval-augmented setup, documents are indexed and relevant chunks are retrieved at query time [14]; this is useful, but each subtle question asks the model to rediscover and re-synthesize the same source fragments. The LLM Wiki pattern instead treats the knowledge base as a persistent, compounding artifact: raw sources are preserved, the model maintains human-readable Markdown pages, and later queries read the already-organized wiki rather than starting from raw chunks each time [13]. Inno Agent adopts this design for educational content: L2 stores learning materials, source summaries, entities, concepts, and later analyses, while learner ability, goals, preferences, and misconceptions remain in L1.

The main design choice is to separate evidence, synthesis, and navigation. Evidence is preserved as raw material; synthesis is written as readable wiki pages; navigation is maintained through an index and a chronological log. This gives the learner a memory that is not only retrievable by the agent but also browsable, auditable, and correctable as ordinary Markdown content.

6.2 Content Organization and Page Schema

L2 is organized as a small knowledge workspace rather than a hidden database. Raw uploaded files, web captures, conversations, and research outputs are preserved as source evidence; extracted text provides a faithful Markdown view of those sources; and the wiki contains the learner-facing knowledge pages. Four page categories give the memory a stable content model: `source-summary` pages explain individual materials, `entity` pages collect information about named artifacts such as papers, projects, libraries, or people, `concept` pages describe reusable ideas and mechanisms, and `analysis` pages are reserved for comparisons, learning routes, and durable conclusions.

Each page carries a small YAML frontmatter that makes the wiki navigable and reviewable. The most important fields are the title, page type, tags, source links, source ids, update date, review status, and confidence. A typical source page has the following shape:

```
---
title: Attention Is All You Need
```



(a) Querying L2 from the chat while browsing the linked concept graph.



(b) A source-summary page generated from an archived paper, with tags, summary, key concepts, and takeaways.

Figure 8: Native Wiki knowledge memory in the Web UI. L2 is exposed as both an agent-queried memory and a learner-facing wiki: the learner can ask questions against archived materials, inspect the concept graph, and open source-summary pages that preserve the synthesized content behind an answer.

```
created: 2026-06-04
type: source-summary
tags: [source-summary, transformer]
sources:
  - raw/uploads/2026-06-04-attention-is-all-you-need-a1b2c3.pdf
source_ids:
  - l2src_7f3a9c21
updated: 2026-06-04
status: draft
confidence: medium
---
# Attention Is All You Need

## Summary

1-3 paragraphs summarizing the source.

## Key Concepts

- [[Transformer]]: sequence model based on attention.
- [[Self-Attention]]: mechanism for token-token interaction.

## Takeaways

- Key takeaway grounded in this source.
- Another takeaway useful for later tutoring.

## Source

Full extracted text: ‘extracted/attention-is-all-you-need-d4e5f6.md‘
```

Reusable concept and entity pages follow the same contract, but their body is organized around definitions and related sources rather than a one-document summary:

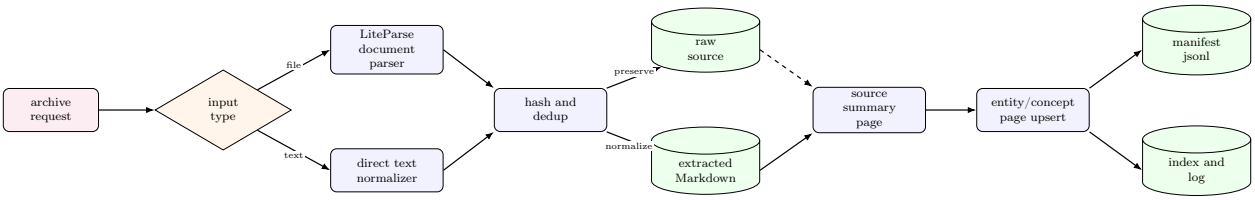


Figure 9: L2 ingestion pipeline. The archive tool normalizes text and document inputs, rejects duplicate content by hash, preserves original evidence, writes extracted Markdown, creates a source-summary page, maintains linked entity/concept pages, and then updates the manifest, index, and log.

```
---
title: Self-Attention
created: 2026-06-04
type: concept
tags: [concept, transformer]
sources:
  - wiki/sources/attention-is-all-you-need-3a9c21.md
source_ids:
  - l2src_7f3a9c21
updated: 2026-06-04
status: draft
confidence: medium
---
# Self-Attention

## Definition

Self-attention lets each token attend to other tokens in the same sequence.

## Related Sources

- [[Attention Is All You Need]] - ‘wiki/sources/attention-is-all-you-need-3a9c21.md‘
```

This template is intentionally simple: the learner can inspect it directly, while the agent can use the metadata to find relevant pages, trace claims to source material, and avoid mixing learning content with learner-state facts stored in L1.

6.3 Ingestion Pipeline

Archiving is the write path into L2. From a learner's perspective, the agent can archive notes, Markdown documents, conversation excerpts, PDFs, Office documents, slide decks, spreadsheets, and common image formats. Document parsing is handled through LiteParse [25]; if no text can be extracted, the material is rejected with an explicit error instead of becoming an empty memory entry.

The ingestion workflow has three user-visible effects. First, it preserves the original material and extracted text as evidence. Second, it creates a source summary that captures the main ideas, key concepts, and useful takeaways. Third, it updates concept and entity pages so repeated materials gradually accumulate into a richer course-specific knowledge base. Duplicate content is filtered by content hash, and every successful archive updates the wiki index and log so future queries can locate the new material.

The result is not just a file upload record. Each archived source becomes part of a maintained learning wiki: a source page explains the material, concept/entity pages connect it to prior materials, and the index/log make the growth of the knowledge base visible.

6.4 Query Pipeline

Querying L2 starts from the wiki index rather than from raw files. An empty query gives the learner an overview of what the wiki currently contains; a focused query returns relevant source, concept, and entity pages for the agent to synthesize. Answers are expected to cite wiki pages using `[[page name]]` references, making it possible for the learner to inspect the supporting page and continue browsing related materials.

6.5 LLM-Assisted Maintenance

L2 uses the model as a wiki maintainer rather than only as a retriever. During ingestion, the model summarizes the source and proposes linkable concepts or entities; when model output is unavailable, the system falls back to conservative text-based behavior so that archiving can still complete. The current release supports source summaries, linked concept/entity pages, index rebuilding, and logging. More advanced wiki health checks, such as detecting stale claims, orphan pages, or explicit contradictions, remain future work.

6.6 Why a Wiki, Not Only a Vector Store

A wiki is human-readable, reviewable, and editable, which matches the requirement that educational knowledge be traceable and maintainable rather than only similarity-retrievable. The page types give knowledge a natural organization—sources, entities, concepts, analyses—and the index/log files let both the learner and the agent see how the memory has evolved. Compared with raw RAG, the advantage is not that the current search algorithm is more powerful; it is that useful structure is compiled into durable pages during ingestion, so later tutoring turns can read a maintained concept or source page rather than reconstructing the same synthesis from chunks. This makes L2 a complementary memory to L1: L1 answers what the learner is trying to do and where they struggle, while L2 answers what the archived materials say and how concepts are connected.

7 Model Training and Post-Training Results[†]

This section reports an initial supervised post-training run rather than a final model release. The current evidence is intended to test whether an Inno-specific data mixture can make a Qwen3.6 35B-A3B model more efficient for learning-agent use while preserving broad benchmark capability.

7.1 Three-Stream Training Mixture

The Qwen-Inno training mixture combines three complementary sources of supervision. The first stream is educational data extracted from papers and learning materials. It turns educational content into structured examples for concept explanation, misconception diagnosis, hinting, exercise generation, answer feedback, learning-plan construction, and review. This stream teaches the model *how to teach*: it emphasizes pedagogical intent, difficulty calibration, and learner-facing clarity rather than only answer correctness.

The second stream is general high-level chain-of-thought data distilled from Claude Opus. Its role is to preserve transferable reasoning ability: decomposing hard questions, following constraints,

writing code, and solving general benchmark tasks. This stream prevents the model from becoming a narrow tutoring template and keeps it competent on the non-educational tasks that appear inside a learning session, such as math derivations, programming exercises, and tool-oriented problem solving.

The third stream consists of de-identified trajectories from the real Inno Agent system. These traces capture the behavior that cannot be learned from generic reasoning data alone: reading the learner profile, using the compact context pack, archiving materials into the L2 wiki, querying maintained wiki pages, creating Practice Lab workspaces, interpreting terminal runs, and scheduling review jobs. Because these trajectories are expressed in the same tool surface described in table 3, they teach both the educational decision policy and the concrete action policy needed by the deployed agent.

7.2 Distillation Target

The base model is **Qwen3.6-35B-A3B**. We train a LoRA-based checkpoint on the three-stream mixture and refer to the resulting model as **Qwen-Inno-35B-v1**. The design goal is not to maximize benchmark scores by adding more reasoning tokens. Instead, the goal is to obtain an educational-agent model that remains close to the base model on general capability, improves on the education-oriented evaluation signal, and produces shorter reasoning traces when long deliberation is unnecessary.

7.3 Benchmark Capability

Figure 10 summarizes the updated benchmark and internal evaluation log for the base model and Qwen-Inno-35B-v1. edu-paper-qa is our own test set built from educational papers; it is used here as a private education-oriented evaluation signal and has not yet been publicly released. The result remains mixed but encouraging for the intended use case. Qwen-Inno matches AIME25, improves on IF-bench, MMMU-Pro, and edu-paper-qa, and gives up some score on MMLU-Pro, MMLU-Redux, IF-Eval, MMMU, RealWorldQA, MMBenchEN-DEV-v1.1, and OCRBench. We therefore do not claim a uniform benchmark improvement. The more conservative conclusion is that the post-trained model keeps a comparable overall capability profile while shifting the model toward educational behavior and selected reasoning benchmarks.

7.4 Reasoning-Length and Efficiency

For deployment, the important question is not only whether the model answers correctly, but how many tokens it spends before producing a useful answer. We therefore compare the median length of the model output and the explicit `think` segment on AIME, MMLU-Pro, HumanEval, and IFBench. As shown in fig. 11, Qwen-Inno-35B-v1 uses substantially shorter reasoning traces on three of the four evaluated datasets: median output length decreases by 31.8% on AIME, 55.1% on MMLU-Pro, and 72.2% on HumanEval. Since most generated tokens are in the `think` segment rather than the final answer, this reduction directly translates into lower decoding cost and shorter user-visible latency under the same serving stack.

IFBench is the main exception: Qwen-Inno-35B-v1 reasons longer and has more max-token truncations on that set. This suggests that instruction-heavy prompts still need targeted filtering or preference optimization so the model learns when to stop deliberating. Nevertheless, the overall pattern supports the central efficiency hypothesis of this report: after incorporating educational data, general Opus-distilled reasoning data, and real Inno trajectories, the model can stay broadly competitive while becoming more concise on common reasoning and coding-style tasks. For a

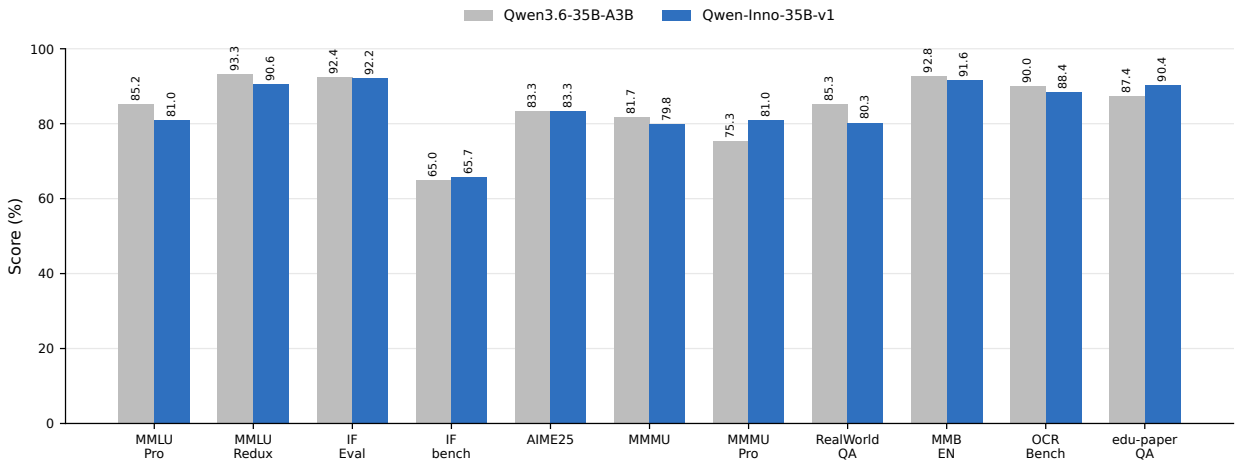


Figure 10: Updated post-training scores for the Qwen3.6-35B-A3B base model and Qwen-Inno-35B-v1. Higher is better.

personal learning agent, this is a practical gain because shorter chain-of-thought segments reduce cost and latency without requiring changes to the memory, tool, or channel architecture.

8 System Integration

8.1 Model Interface

A model is plugged into Inno through configured providers. `InnoConfig` (`src/config.ts`) declares a default provider and model and a set of OpenAI-compatible or Anthropic-style providers, each with model entries carrying `contextWindow` (default 128k) and `maxTokens` (default 8192). The extension registers these providers with the SDK at startup and persists the user's model selection on the `model_select` event. The same interface accepts a hosted general LLM or a locally served distilled Inno model without code changes.

8.2 Tool Use

Before each turn, `before_agent_start` assembles the system prompt from the Inno base prompt, the formatted context pack, any per-workspace `agent.md` and private `.skills`, and the tail of the most recent run record, then appends the SDK's own prompt. The registered tools the model can call are listed in table 3.

8.3 Scaffolding for Smaller Models

> A small model does not need to hold the learner's entire history and knowledge in context. Inno Agent's system memory, tools, and context pack provide external structure, so the model can complete high-quality personalized teaching with far fewer tokens.

This is the integration-level statement of the central hypothesis: the per-turn prompt carries a short, decision-ready context pack and a fixed tool surface, so model size can be traded down without asking the model to reconstruct state it was never given.

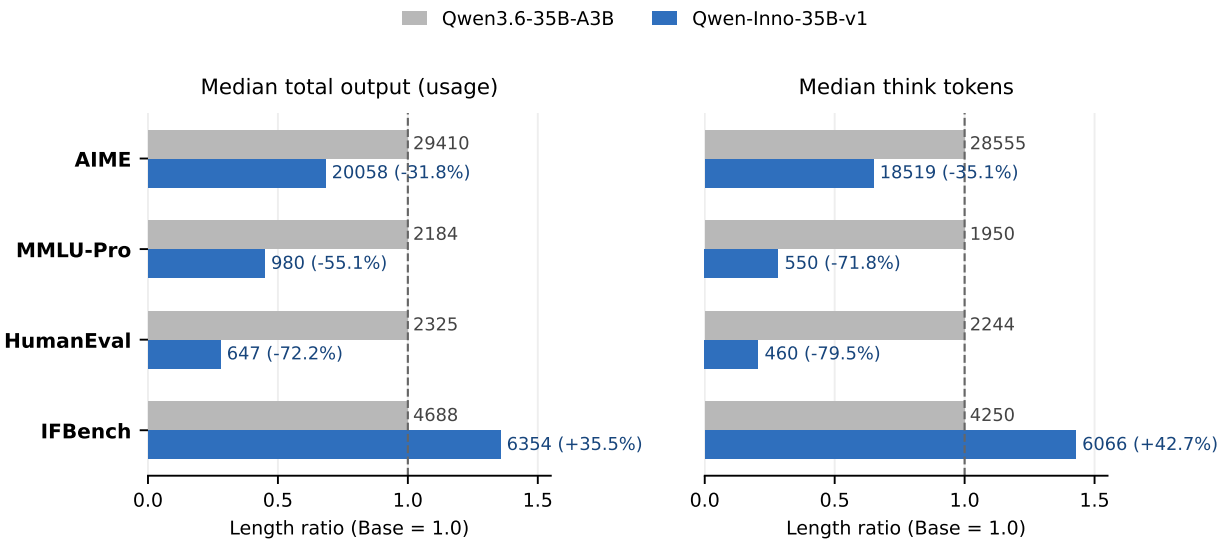


Figure 11: Median length ratios from the original evaluation log. The left panel uses platform-reported total output tokens (`output tokens (usage)`), and the right panel uses the parsed `think` segment; all values are medians rather than means or best cases. Percentages are computed as (Inno − Base)/Base for the metric shown in that panel.

9 Interaction Channels

Personal learning is fragmented across devices and moments: a learner may ask a quick question from a phone, forward a document from an IM client, review a reminder on a work laptop, or capture a confusing screenshot while commuting. Inno Agent treats these device-level touchpoints as first-class learning inputs rather than as separate products. As shown in fig. 13, the same WeChat learning turn can begin on a mobile device and remain visible in the desktop learning workbench. The channel layer lets knowledge acquisition happen wherever the learner already is, while routing each fragment back into the same learner profile, L2 wiki, session records, and agent runtime.

9.1 Unified Channel Interface

All channels implement one interface (`src/channels/channel.ts`), with real-time channels adding lifecycle and a message callback:

```
export interface ChatChannel {
  readonly name: string;
  verify(req): Promise<boolean>;
  parse(body): Promise<IncomingMessage | null>;
  reply(message, text): Promise<void>;
  push(target: PushTarget, text): Promise<void>;
}
export interface RealtimeChatChannel extends ChatChannel {
  onMessage(handler): void;
  start(): Promise<void> | void;
  stop?(): Promise<void>;
}
```



Figure 12: Provider configuration interface. A user can add an OpenAI-compatible provider, set its base URL and API key, register model identifiers and display names, specify context-window and generation-token limits, and mark the model as the default provider for Inno Agent.

Table 3: Tools registered with the Pi runtime (`inno-extension.ts`).

Group	**Tools**
L1 learner	`get_learner_context`, `record_learning_event`, `patch_learner_profile`, `update_learner_profile`, `review_learner_profile`
L2 wiki	`l2_archive`, `l2_query`
Documents	`parse_document`
Scheduler	`create_scheduled_job`, `list_scheduled_jobs`, `update_scheduled_job`, `delete_scheduled_job`, `run_scheduled_job`
Practice Lab	`create_practice_lab`
Interaction	`ask_user_question`

`IncomingMessage` normalizes channel, message/chat/user ids, text, and attachments; `PushTarget` is a `(channel, chatId)` pair used for proactive pushes.

9.2 Personal Dispatcher

`PersonalChannelDispatcher` (`personal-dispatcher.ts`) is the single funnel for fragmented learning traffic. It deduplicates by `(channel, messageId)` with a 24-hour TTL store, drops empty messages, records the current chat as the channel's default push target, and recognizes `/new` (and Chinese equivalents) to start a fresh session. It converts image attachments to base64 for the model, prefixes each prompt with a source-channel tag so the agent can pick the right reminder channel, truncates text to 20k characters, and appends downloaded file paths. It resolves a dedicated session per chat key so channel conversations stay isolated from the web session, but every channel turn still reaches the same learning memory stack and serialized agent loop. This lets a phone message, a Feishu file, a WeChat follow-up, and a web-session question accumulate into one coherent personal learning history rather than four disconnected conversations. Each run is logged with status and duration; failed replies are retried once.



Figure 13: Cross-device interaction channels. The mobile WeChat conversation provides a low-friction entry point for fragmented learning, while the desktop Inno Agent workbench shows the same channel message connected to the agent trace, provider settings, and learning workspace.

9.3 Native and Bridge Channels

Feishu is integrated natively over a Lark WebSocket long connection (`@larksuiteoapi/node-sdk`), parsing text, post, image, and file messages and segmenting long replies to respect post size limits. The bridge server is an extension point for sidecar-based channels: an external process can post a `BridgeMessageBody` to `/api/bridge/messages` with a `Bearer` token, and the backend can call back to the sidecar's `/reply` and `/push`; however, QQ is not a supported channel in the current release because no QQ sidecar adapter is provided. WeChat is experimental, implemented over the iLink polling client with QR-code login; WeCom is a stub. Both Feishu and WeChat support a private-chat-only mode and an allowed-user-ID whitelist.

9.4 Design Trade-offs

The channel layer is deliberately not a customer-service or group-chat system: there is no group messaging, no multi-party session isolation, and no multi-tenant routing. The design optimizes for one learner acquiring knowledge through many devices, not for many users sharing one bot. Every personal entry point maps to the same single learner context, which is the simplification that lets the learner profile and wiki remain singular and coherent even when learning happens in short, device-mediated fragments.

10 Proactive Learning Scheduler

Figure 14 shows a concrete scheduling flow: the learner asks for a near-term review reminder in WeChat, the scheduler later pushes the generated review prompt back through the same channel, and the desktop workbench exposes the corresponding job for inspection and editing.

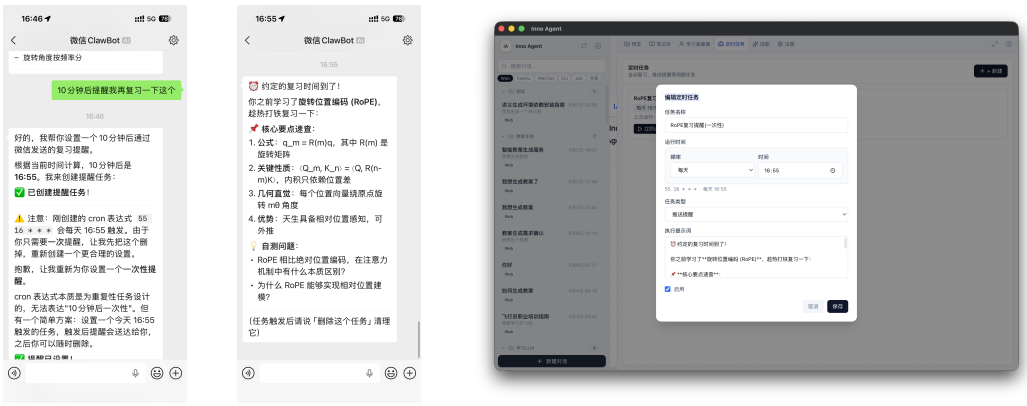


Figure 14: Natural-language reminder creation, scheduled WeChat review delivery, and desktop job inspection. From left to right: the agent converts a "review in ten minutes" request into a reminder, later pushes a concise RoPE review card, and exposes the same job in the desktop scheduler workbench for inspection and editing.

10.1 Job Model

A `ScheduledJob` (`src/scheduler/types.ts`) carries an id, name, cron expression, timezone, enabled flag, optional channel and push target, a `taskType`, the prompt to run, run/next timestamps, last status and error, and run/failure counters. Jobs are stored in `jobs.json` and run records are appended to `runs.jsonl`.

10.2 Built-in Learning Jobs

The `taskType` enumeration is fixed in code:

```
type TaskType =
  | "daily_review" | "weekly_summary" | "graphify_update"
  | "learner_profile_reflection" | "spaced_review"
  | "push_reminder" | "custom_prompt";
```

These cover daily review, weekly summary, knowledge-base (graphify) update, learner-profile reflection, spaced review, push reminders, and arbitrary custom prompts.

10.3 Execution Flow

The scheduler is an in-process loop that performs an initial check after 5 seconds and then scans all jobs every 60 seconds. Each tick loads normalized jobs from the store, skips disabled jobs and jobs already present in an in-memory running set, and uses `cron-parser` to decide whether the previous scheduled fire time has not yet been run. Execution first marks the job as `running`. A `push_reminder` formats the stored prompt directly and appends it as an assistant notification; other task types execute the prompt through the serialized agent runner, so they use the same L1/L2/L3 context path as an ordinary turn. The runner then resolves a push target from the configured channel or the channel default, attempts delivery, appends a JSONL run record with status, duration, output preview, trigger, and push outcome, and updates `lastRunAt`, `nextRunAt`, `lastStatus`, `lastError`, `runCount`, and `failureCount`. A one-shot cron (literal minute, hour, day-of-month, and month) is disabled after it fires, which is how "remind me tomorrow at 14:30" avoids becoming a yearly recurring job. The implementation-level flow is summarized in fig. 15.

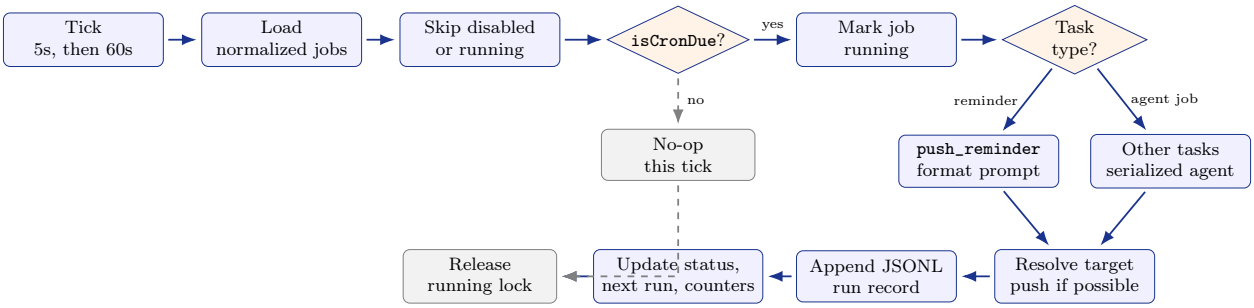


Figure 15: Implementation-level scheduler execution flow. Enabled jobs that are due are marked running, dispatched either as direct reminders or serialized agent prompts, optionally pushed to a channel, recorded in the run log, and then used to update the job state and release the in-memory running lock.

10.4 Natural-Language Job Creation

Because job creation is itself a tool (`create_scheduled_job`), a request such as "summarize what I learned today at 10 pm every night and send it to Feishu" is realized by the agent *creating a scheduled job*, not by promising verbally. The system prompt further requires that `push_reminder` jobs specify a channel, defaulting to the message's source channel and otherwise asking the user via `ask_user_question`.

11 Practice Lab and Web Terminal

11.1 Motivation

Learning to program or to run experiments requires doing, not only reading. The Practice Lab closes the loop from writing code, to running it, to observing output, to analyzing errors, to revising and reflecting—inside the same workbench as the conversation.

11.2 Workspace-Centered UI

The web app (`web/src/react/App.tsx`) is a three-column React 19 workbench—session navigation, chat, and a workspace panel—with responsive collapse breakpoints. The workspace panel hosts tabs for file preview, the wiki, a knowledge graph (Cytoscape), jobs, settings, skills, and the learner profile. It renders Markdown/HTML/PDF/image/code previews, edits with CodeMirror, and embeds an xterm.js terminal drawer with a run-records list.

11.3 Practice Lab Workflow

The agent never silently runs commands. It writes a lab via `create_practice_lab` (validating that each file path stays inside the session's workspace), returns a `mainFile` and a `suggestedCommand`, and the UI opens the main file and shows a Run button. The user clicks Run, the terminal streams output, a run record is saved, and a follow-up question is answered using the injected tail of that run record.

Figure 16 illustrates this loop in the current workbench. The agent can generate runnable code directly into the learning workspace, the learner can execute it in the embedded terminal, and the resulting run record can be archived back into the L2 wiki memory as reusable learning evidence rather than remaining an ephemeral console transcript.

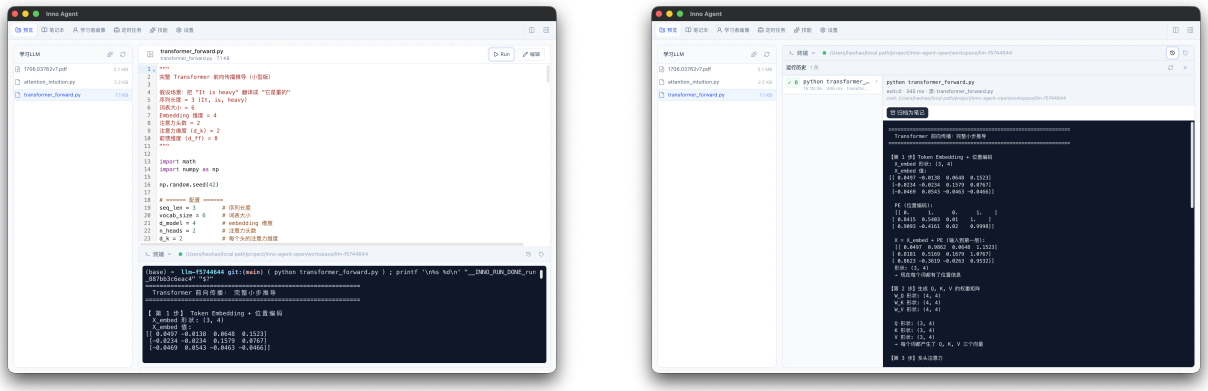


Figure 16: Practice Lab execution loop. The left screenshot shows generated code opened beside the embedded terminal, where the learner runs the suggested Python command and sees streamed output. The right screenshot shows the persisted run record, including the control for archiving the terminal result into the L2 wiki memory.

```
user asks for a demo -> agent writes files (create_practice_lab)
-> workspace opens main file -> user clicks Run -> terminal shows output
-> run record saved -> optional L2 archive -> agent explains from run record
```

11.4 Terminal Backend

The terminal is a `node-pty` process per session. The backend selects `$SHELL` (falling back to `/bin/bash` or PowerShell), spawns an `xterm-256color` PTY, and starts it in the workspace directory resolved for the current session. Commands run wrapped in a subshell terminated by a sentinel (`__INNO_RUN_DONE_<runId> <exitCode>`) sent under bracketed-paste mode, which lets the manager detect completion and exit code reliably and strip the marker from forwarded output. Output is persisted ANSI-stripped to `runs/YYYY-MM-DD/<runId>.{json,log}`, with a 256 KB tail cap on reads. A planned evolution adds container and Kubernetes-exec backends for per-lab isolation.

11.5 Safety Principles

Several safeguards are implemented in code, while process isolation remains an explicit deployment requirement. The `create_practice_lab` tool writes only after resolving each requested file against the session workspace and rejecting paths whose relative form escapes that root; the workspace registry applies the same root check when resolving workspace IDs. A successful lab tool call only opens the generated main file and displays a Run button, while actual execution is triggered by the user's click and sent as a bounded terminal `run` event. The PTY starts from the resolved workspace directory, records each run with stripped terminal control sequences, and sanitizes the child environment by removing variable names containing `API_KEY`, `APIKEY`, `SECRET`, `TOKEN`, or `PASSWORD`. This is path and environment hygiene, not a full sandbox: production deployments should still run terminals in containers or Kubernetes-exec backends and avoid mounting sensitive host paths or the Docker socket.

12 Session and Workspace Scoping

Inno Agent separates the conversational session from the working directory in which learning artifacts live. The backend keeps a session-to-workspace map: when a session is opened, its workspace id is resolved by the workspace registry into an absolute directory under the configured workspace root, and both agent execution and terminal creation use that binding. New named workspaces are created as slug-plus-hash directories to avoid collisions, while channel-originated sessions can be attached to stable channel workspaces such as the WeChat workspace. This design lets a learner keep different learning projects separate without duplicating the long-term learner profile or the L2 wiki. Figure 17 summarizes how the runtime assembles this scoped context.

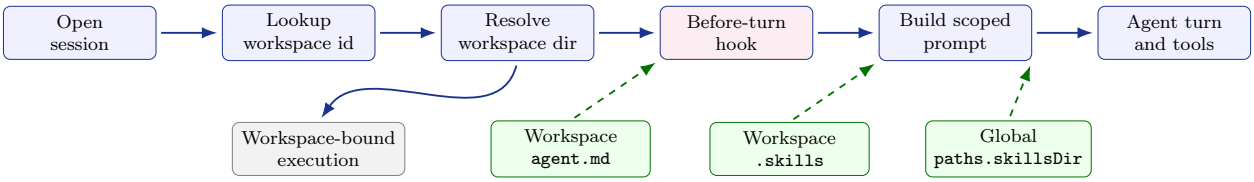


Figure 17: Session and workspace scoping flow. A session is bound to a resolved workspace directory; before each agent turn, the runtime hook injects workspace-local instructions from `agent.md`, workspace-private skills from `.skills`, and globally installed skills from `paths.skillsDir`. The same resolved workspace also anchors terminal and Practice Lab execution.

Each workspace can also carry local instructions. Before every agent turn, the Inno extension resolves the active session's workspace and, if `agent.md` exists at the workspace root, injects its content as an additional workspace-context section. This file is the lightweight mechanism for task-local behavior: a workspace for lesson-plan design, for example, can specify how to analyze curriculum goals, structure objectives, include classroom activities, and evaluate student understanding. These instructions stay local to that workspace instead of becoming global policy for all future learning sessions.

Skills follow the same two-level scoping rule. Global skills are installed under the configured `paths.skillsDir` and loaded by the Pi resource loader for all sessions; they are appropriate for reusable capabilities such as manipulating Office files, parsing common document formats, or performing general research workflows. Workspace-private skills live under `<workspace>/.skills`; the workspace browser can install a `.zip` or `.md` skill package into that directory, and the extension discovers those skills before each turn and formats them into the prompt for the active workspace only. Local skills are therefore useful for project-specific pedagogy—for instance, a skill that teaches the agent how to produce a better lesson plan for a particular course—while global skills remain shared utilities available across all workspaces.

13 Conclusion and Future Work

This report presents Inno Agent as an early exploration of the educational-agent form: a personal learning system in which learner modeling, knowledge memory, workspace-local teaching procedures, practice tooling, and model post-training are designed together rather than treated as separate add-ons. The current implementation and the preliminary Qwen-Inno-35B-v1 checkpoint are publicly released on the title page. These releases are intended to make the system direction inspectable and reproducible, while making clear that the project is still at an initial stage.

The main conclusion is therefore conservative. Inno Agent suggests that educational agents

should be built as instructional systems, not only as chat interfaces over large models. The Dick and Carey framing helps organize this claim: learner goals, learner/context analysis, content analysis, performance objectives, instructional strategies, materials, assessment, and formative revision must be connected in one operational loop. In the current system, L1 learner memory, L2 wiki memory, workspace skills, `agent.md`, scheduled review, and Practice Lab traces provide a first implementation of that loop. However, the educational-theory grounding, learner-modeling methods, and learning-outcome evaluation are not yet sufficiently systematic.

Future work will focus on closing the loop among model post-training, educational-agent development, and empirical education research. On the model side, we will improve the training mixture with higher-quality educational trajectories, preference data, and targeted optimization for when the model should explain, hint, ask, assess, or stop reasoning. On the agent side, we will make learner profiling, wiki maintenance, learning-path planning, and workspace skills more robust and transparent. On the education-research side, we will move beyond benchmark and length analysis toward classroom or learner studies, including multi-subject tasks, long-term A/B studies, learning-outcome measurement, and safety evaluation. The goal is to turn Inno Agent from an open-source prototype into a systematic research platform for studying how post-trained models, agent tools, and instructional design can jointly support long-term personalized learning.

References

[1] Benjamin S. Bloom. The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring. *Educational Researcher*, 13(6):4–16, 1984.

[2] R. Böck et al. Learner models: Design, components, structure, and modelling. *User Modeling and User-Adapted Interaction*, 2025.

[3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, 2020.

[4] Susan Bull. Negotiated learner modelling to maintain today's learner models. *Research and Practice in Technology Enhanced Learning*, 2016.

[5] Susan Bull and Judy Kay. SMILI: A framework for interfaces to learning data in open learner models, learning analytics and related fields. *International Journal of Artificial Intelligence in Education*, 26:293–331, 2016.

[6] Nicholas J. Cepeda, Harold Pashler, Edward Vul, John T. Wixted, and Doug Rohrer. Distributed practice in verbal recall tasks: A review and quantitative synthesis. *Psychological Bulletin*, 132(3):354–380, 2006.

[7] Albert T. Corbett and John R. Anderson. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4:253–278, 1994.

[8] Walter Dick, Lou Carey, and James O. Carey. *The Systematic Design of Instruction*. Pearson, Boston, MA, 8 edition, 2015.

[9] Earendil Works. Pi: AI agent toolkit and agent harness. `https://github.com/earendil-works/pi/tree/main`, 2026. GitHub repository.

[10] Robert M. Gagné. *The Conditions of Learning and Theory of Instruction*. Holt, Rinehart and Winston, New York, 4 edition, 1985.

[11] Kostas Hatalis et al. Memory matters: The need to improve long-term memory in LLM-agents. In *AAAI Spring Symposium*, 2024.

[12] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, 2024.

[13] Andrej Karpathy. LLM Wiki: A pattern for building personal knowledge bases using LLMs. `https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f`, 2026. GitHub Gist.

[14] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020.

[15] Qi Liu et al. A survey on deep learning based knowledge tracing. *Knowledge-Based Systems*, 2022.

[16] Yanjin Long and Vincent Aleven. Enhancing learning outcomes through self-regulated learning support with an open learner model. *User Modeling and User-Adapted Interaction*, 2017.

[17] M. David Merrill. First principles of instruction. *Educational Technology Research and Development*, 50(3):43–59, 2002.

[18] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022.

[19] Charles Packer et al. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[20] Ernesto Panadero. A review of self-regulated learning: Six models and four directions for research. *Frontiers in Psychology*, 8:422, 2017.

[21] Zachary A. Pardos and Neil T. Heffernan. Modeling individualization in a bayesian networks implementation of knowledge tracing. In *User Modeling, Adaptation, and Personalization*, 2010.

[22] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the ACM Symposium on User Interface Software and Technology*, 2023.

[23] Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas J. Guibas, and Jascha Sohl-Dickstein. Deep knowledge tracing. In *Advances in Neural Information Processing Systems*, 2015.

[24] Robert A. Reiser. A history of instructional design and technology: Part II: A history of instructional design. *Educational Technology Research and Development*, 49(2):57–67, 2001.

[25] Run Llama. LiteParse: Document parsing for LLM applications. `https://github.com/run-llama/liteparse`, 2026. GitHub repository.

[26] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

[27] Kurt VanLehn. The behavior of tutoring systems. *International Journal of Artificial Intelligence in Education*, 16(3):227–265, 2006.

[28] Kurt VanLehn. The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational Psychologist*, 46(4):197–221, 2011.

[29] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.

[30] T. Wu et al. LongMemEval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.

[31] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.